

# Cortical Column Voting Simulation

- **Name:** Hisham D Macaraya
- **Course:** ITAI 4374 Neuroscience as a Model for AI
- **Houston City College**
- **Professor:** Patricia Mcmanus
- **Date:** January 28, 2026

## Part 1: Introduction

### 1.1 What This Lab Demonstrated

In this lab, we implemented a simplified simulation of cortical column voting based on **Jeff Hawkins' Thousand Brains Theory**. This theory proposes that the brain learns by building a model of the world using thousands of simultaneously operating cortical columns, each learning complete models of objects, parts of objects, and environments. Our simulation demonstrated:

- **Multiple "columns"** that each received partial and noisy sensory input, mimicking how individual cortical columns in the brain might receive limited information about a perceived object.
- A **voting mechanism** where these independent columns shared their beliefs about the identity of the perceived object with their neighbors.
- **Consensus** emerging from this distributed processing, illustrating how collective intelligence can overcome individual uncertainty.
- **Visualization** of how agreement developed over time, providing insight into the dynamic process of belief convergence.

### 1.2 The Algorithm

Our simulation followed these steps:

| Step             | What Happened                                | Implemented Function                |
|------------------|----------------------------------------------|-------------------------------------|
| 1. Setup         | Created N columns, defined possible objects  | Basic initialization                |
| 2. Sensory Input | Each column received partial, noisy evidence | <code>create_sensory_input()</code> |
| 3. Initialize    | Set initial beliefs from evidence            | <code>initialize_beliefs()</code>   |
| 4. Vote          | Columns shared beliefs with neighbors        | <code>voting_round()</code>         |
| 5. Iterate       | Repeated voting until consensus              | <code>run_voting()</code>           |
| 6. Visualize     | Showed how consensus emerged                 | <code>visualize_consensus()</code>  |

## 1.3 Key Concepts Explored

- **Distributed Representation:** No single column knew the answer; knowledge emerged from many columns working together. Unlike hierarchical models, here knowledge is distributed across peers.
- **Lateral Connections:** Columns communicated with neighbors, sharing their current beliefs. This simulates the long-range lateral connections found in the neocortex.
- **Consensus:** Through iterative voting, columns converged on a shared interpretation. This robustness allows the brain to recognize objects even with partial or occluded data.

## ✓ Part 2: Environment Set-up

```
#Initial Set-up
import numpy as np
import matplotlib.pyplot as plt

# Set seed for reproducibility
np.random.seed(42)

# Define objects our system can recognize
OBJECTS = ["cup", "ball", "book", "phone", "apple"]
NUM_OBJECTS = len(OBJECTS)
```

```
print("Setup complete!")
print("Objects:", OBJECTS)
```

```
Setup complete!
Objects: ['cup', 'ball', 'book', 'phone', 'apple']
```

## ✓ Part 3: Implementation

### ✓ 3.1 Sensory Input

```
def create_sensory_input(true_object, num_columns, noise_level=0.3):
    """
    Creates evidence matrix for all columns.

    Parameters:
    - true_object: int, index of the actual object (0 to 4)
    - num_columns: int, number of columns to simulate
    - noise_level: float, amount of random noise (0 to 1)

    Returns:
    - evidence: numpy array of shape (num_columns, NUM_OBJECTS)
    """

    # Create matrix filled with random noise
    # We initialize the evidence matrix with random background noise.
    # This simulates uncertain sensory information before any strong signal is detected.
    evidence = np.random.uniform(low=0, high=noise_level, size=(num_columns, NUM_OBJECTS))

    # Add signal for the true object
    # For the object that is actually present (true_object), we add a stronger signal.
    # This mimics the sensor picking up features of the specific object.
    for col in range(num_columns):
        signal = np.random.uniform(0.5, 1.0)
        evidence[col, true_object] += signal

    # Normalize each row to sum to 1
```

```
# We normalize the evidence so that the values in each column (representing probabilities or activation levels)
# sum up to 1 across all possible objects. This makes the input compatible with probabilistic models.
row_sums = np.sum(evidence, axis=1, keepdims=True)
evidence = evidence / row_sums

return evidence
```

```
# Test create_sensory_input
true_object = 0 # "cup"
num_columns = 6
evidence = create_sensory_input(true_object, num_columns, noise_level=0.3)
print(f"True object: {OBJECTS[true_object]}")
print(f"Evidence shape: {evidence.shape}")
print("\nEach column's evidence:")
for i in range(num_columns):
    best = OBJECTS[np.argmax(evidence[i])]
    conf = np.max(evidence[i]) * 100
    print(f" Column {i}: {best} ({conf:.1f}%)"
```

```
True object: cup
Evidence shape: (6, 5)
```

```
Each column's evidence:
Column 0: cup (62.1%)
Column 1: cup (65.2%)
Column 2: cup (48.6%)
Column 3: cup (54.9%)
Column 4: cup (76.1%)
Column 5: cup (53.0%)
```

## ✓ 3.2 Initialize Beliefs

This simple function sets each column's initial beliefs equal to its sensory evidence.

**What the function should do:**

- Take the `evidence` matrix as input.
- Return a **copy** of the `evidence` matrix (to ensure the original `evidence` remains unchanged).

```
def initialize_beliefs(evidence):  
    """Initialize beliefs from sensory evidence."""  
    return np.copy(evidence) # Return a copy of evidence
```

```
beliefs = initialize_beliefs(evidence)  
print("Beliefs initialized!")  
print(f"Shape: {beliefs.shape}")  
print(f"Same as evidence? {np.allclose(beliefs, evidence)}") # Should be True
```

```
Beliefs initialized!  
Shape: (6, 5)  
Same as evidence? True
```

### ✓ 3.3 The Voting Mechanism

This is the core of the simulation! Each column updates its beliefs by combining its own evidence with votes from other columns.

#### The Formula:

For each column, the new belief is:  $\text{new\_belief} = (1 - \text{vote\_weight}) \times \text{own\_evidence} + \text{vote\_weight} \times \text{average\_neighbor\_votes}$

- $\text{vote\_weight} = 0.3$  means 70% trust in own evidence, 30% trust in neighbors.
- $\text{average\_neighbor\_votes}$  = mean of all **OTHER** columns' current beliefs.

What the function should do:

1. Create an empty matrix for new beliefs.
2. For each column:
  - Calculate own contribution:  $(1 - \text{vote\_weight}) \times \text{evidence}[\text{col}]$
  - Get votes from other columns (exclude current column).
  - Calculate vote contribution:  $\text{vote\_weight} \times \text{mean}(\text{other\_beliefs})$ .

- Add them together.
3. Normalize each row to sum to 1.
  4. Return the new beliefs.

```
def voting_round(beliefs, evidence, vote_weight=0.3):
    """
    One round of voting among columns.

    Parameters:
    - beliefs: current beliefs, shape (num_columns, NUM_OBJECTS)
    - evidence: original sensory evidence (doesn't change)
    - vote_weight: float 0-1, how much to trust neighbors

    Returns:
    - new_beliefs: updated beliefs after voting
    """
    num_columns = beliefs.shape[0]
    new_beliefs = np.zeros_like(beliefs)

    for col in range(num_columns):
        # TODO 1: Calculate this column's own contribution
        # This part represents the column's trust in its own direct sensory evidence.
        # It's weighted by (1 - vote_weight), meaning if vote_weight is 0.3, it trusts its own evidence 70%.
        own_contribution = (1 - vote_weight) * evidence[col]

        # TODO 2: Get beliefs from OTHER columns (not this one)
        # We need to collect the beliefs from all other columns to calculate the average neighbor vote.
        other_indices = [i for i in range(num_columns) if i != col]
        other_beliefs = beliefs[other_indices]

        # TODO 3: Calculate average vote from others
        # The average vote is the mean of the beliefs across all other columns for each object.
        # axis=0 ensures that we average across the columns, resulting in a single row representing the average
        avg_vote = other_beliefs.mean(axis=0)

        # TODO 4: Calculate vote contribution
        # This part represents how much the column trusts the collective opinion of its neighbors.
```

```

# It's weighted by vote_weight, meaning if vote_weight is 0.3, it trusts neighbors 30%.
vote_contribution = vote_weight * avg_vote

# TODO 5: Combine own + vote contributions
# The new belief for the current column is a sum of its own contribution and the neighbors' vote contri
new_beliefs[col] = own_contribution + vote_contribution

# TODO 6: Normalize rows
# After combining contributions, the sums might not be 1. We normalize each row (column's beliefs)
# to ensure they represent a probability distribution, summing to 1 across all objects.
row_sums = new_beliefs.sum(axis=1, keepdims=True)
# Avoid division by zero if a row sum is zero (though unlikely in this model with positive initial evidence)
new_beliefs = new_beliefs / row_sums

return new_beliefs

```

```

# Test voting_round

# Ensure initial_beliefs is available, or re-initialize if needed
# Assuming 'evidence' is already defined from previous steps.
initial_beliefs = initialize_beliefs(evidence)

print("Before voting:")
for i in range(min(num_columns, 3)): # Limiting to first 3 columns for concise output
    print(f" Col {i}: {OBJECTS[np.argmax(initial_beliefs[i])]} ({np.max(initial_beliefs[i])*100:.1f} %)")

new_beliefs = voting_round(initial_beliefs, evidence, vote_weight=0.3)

print("\nAfter 1 voting round:")
for i in range(min(num_columns, 3)): # Limiting to first 3 columns for concise output
    print(f" Col {i}: {OBJECTS[np.argmax(new_beliefs[i])]} ({np.max(new_beliefs[i])*100:.1f} %)")

```

```

Before voting:
Col 0: cup (62.1 %)
Col 1: cup (65.2 %)
Col 2: cup (48.6 %)

```

```

After 1 voting round:
Col 0: cup (61.4 %)

```

```
Col 1: cup (63.4 %)
Col 2: cup (52.7 %)
```

### ✓ 3.4 Run Voting Loop

Now we will implement the `run_voting` function. This function runs multiple voting rounds until columns agree or we hit the maximum iterations.

#### What the function should do:

1. Initialize beliefs from evidence.
2. Store beliefs in a history list (for visualization later).
3. Loop up to `max_rounds` times:
  - Call `voting_round()` to update beliefs.
  - Add new beliefs to history.
  - Check if consensus is reached ( $\geq 90\%$  of columns agree on the same object).
  - If consensus reached, break out of loop.
4. Calculate final result (which object won, with what confidence).
5. Return history, winning object, confidence, and iteration count.

```
# Example: checking consensus
predictions = np.argmax(beliefs, axis=1) # Best guess per column
# predictions might be [0, 0, 0, 2, 0, 0] - most say object 0
# Count votes for each object
vote_counts = np.bincount(predictions, minlength=NUM_OBJECTS)
# vote_counts = [5, 0, 1, 0, 0] - object 0 has 5 votes
# Find most common and calculate agreement
most_common = vote_counts.argmax() # = 0
agreement = (predictions == most_common).mean() # = 5/6 = 0.833
```

```
def run_voting(evidence, max_rounds=20, vote_weight=0.3):
    """
    Runs the voting process until consensus or max rounds.
```



```

Returns:
- history: list of belief matrices
- winner: index of winning object
- confidence: average confidence of winner
- rounds: number of rounds run
"""
# 1. Initialize
current_beliefs = initialize_beliefs(evidence)
history = [current_beliefs.copy()]
num_columns = evidence.shape[0]

print(f"Starting voting with {num_columns} columns...")

rounds_run = 0
for i in range(max_rounds):
    rounds_run += 1
    # 2. Update beliefs
    current_beliefs = voting_round(current_beliefs, evidence, vote_weight)
    history.append(current_beliefs.copy())

    # 3. Check consensus (>= 90% agree)
    # Get top object for each column
    column_votes = np.argmax(current_beliefs, axis=1)
    # Find most common vote
    counts = np.bincount(column_votes, minlength=current_beliefs.shape[1])
    winner_candidate = np.argmax(counts)
    agreement_pct = counts[winner_candidate] / num_columns

    if agreement_pct >= 0.90:
        print(f"Consensus reached at round {i+1}!")
        break
else:
    print("Max rounds reached without full consensus.")

# 4. Calculate final result
final_avg = np.mean(current_beliefs, axis=0)
final_winner = np.argmax(final_avg)
final_conf = final_avg[final_winner]

```

```
return np.array(history), final_winner, final_conf, rounds_run
```

```
# Test run_voting
np.random.seed(42)
evidence = create_sensory_input(true_object=0, num_columns=6,
noise_level=0.3)
history, result, confidence, iters = run_voting(evidence)
print(f"Result: {OBJECTS[result]}")
print(f"Confidence: {confidence*100:.1f}%")
print(f"Iterations: {iters}")
print(f"Correct? {'Yes!' if result == 0 else 'No'}")
```

```
Starting voting with 6 columns...
Consensus reached at round 1!
Result: cup
Confidence: 61.2%
Iterations: 1
Correct? Yes!
```

## ✓ 3.5 Visualization

Finally, we will visualize the consensus. We want to see if the columns eventually agree on the `true_object`.

We will plot the **average confidence** across all columns for the true object over time.

### Visualization Details

Create a visualization showing how consensus emerges over time.

#### What to Plot:

- **Plot 1 (Left): Confidence over time for each object**
  - X-axis: Voting round (0, 1, 2, ...)
  - Y-axis: Average confidence across all columns
  - One line per object

- Make the **TRUE** object's line bold/solid, others dashed
- **Plot 2 (Right): Agreement over time**
  - X-axis: Voting round
  - Y-axis: Fraction of columns that agree
  - Draw a horizontal line at 0.9 (consensus threshold)

```
def visualize_consensus(history, true_object):
    """Visualize how consensus emerges."""

    # Calculate average confidence per object at each iteration
    # history is a list of belief matrices
    # We need: for each time step, average confidence for each object
    avg_confidence = np.array([h.mean(axis=0) for h in history])
    # Shape: (num_iterations, NUM_OBJECTS)

    # TODO 1: Calculate agreement at each time step
    agreement_over_time = []
    for h in history:
        predictions = np.argmax(h, axis=1)
        counts = np.bincount(predictions, minlength=NUM_OBJECTS)
        # Calculate fraction of columns that agree
        agreement = counts.max() / h.shape[0]
        agreement_over_time.append(agreement)

    # Create figure with 2 subplots
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))

    # TODO 2: Plot 1 - Confidence over time
    ax1 = axes[0]
    for obj_idx in range(NUM_OBJECTS):
        # Make true object bold, others dashed
        if obj_idx == true_object:
            style = '-'
            width = 3
        else:
            style = '--'
            width = 1
```

```

        ax1.plot(avg_confidence[:, obj_idx], style,
                 linewidth=width, label=OBJECTS[obj_idx])

    ax1.set_xlabel('Voting Round')
    ax1.set_ylabel('Average Confidence')
    ax1.set_title('Confidence Evolution')
    ax1.legend()
    ax1.set_ylim(0, 1)
    ax1.grid(True, alpha=0.3)

    # TODO 3: Plot 2 - Agreement over time
    ax2 = axes[1]
    ax2.plot(agreement_over_time, 'b-', linewidth=2, marker='o')
    ax2.axhline(y=0.9, color='r', linestyle='--', label='Threshold')
    ax2.set_xlabel('Voting Round')
    ax2.set_ylabel('Column Agreement')
    ax2.set_title('Consensus Emergence')
    ax2.legend()
    ax2.set_ylim(0, 1.05)
    ax2.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig('consensus_plot.png', dpi=150)
    plt.show()

    print("Plot saved as consensus_plot.png")

```

```

# --- Run the Full Simulation ---

```

```

# 1. Run the voting loop

```

```

history, winner, conf, rounds = run_voting(evidence, max_rounds=20, vote_weight=0.3)

```

```

# 2. Visualize the results

```

```

visualize_consensus(history, true_object)

```

```

# 3. Check final agreement

```

```

print(f"\nFinal consensus: {OBJECTS[winner]} ({conf*100:.1f}%")

```

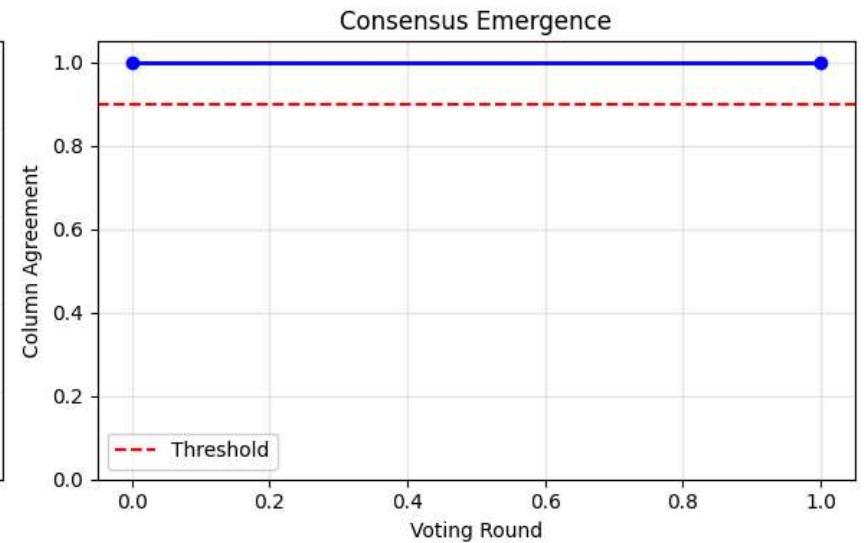
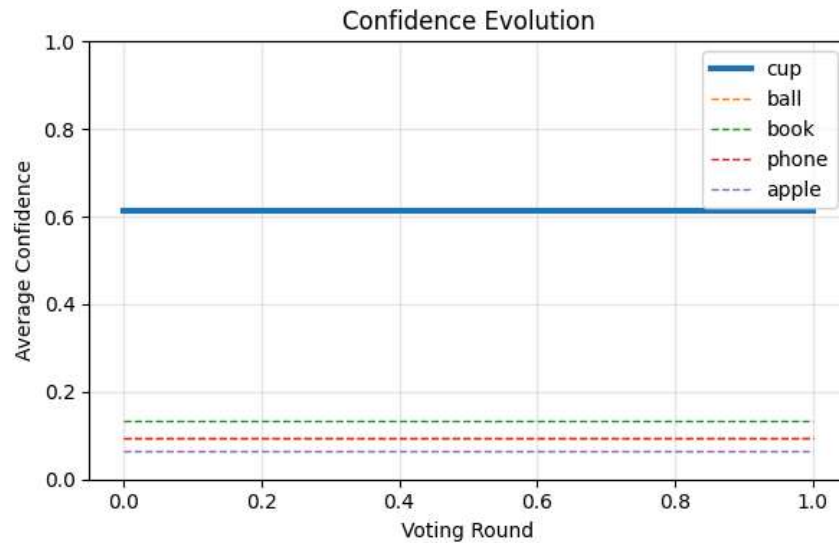
```

print(f"Converged in {rounds} rounds.")

```

Starting voting with 6 columns...

Consensus reached at round 1!



Plot saved as consensus\_plot.png

Final consensus: cup (61.2%)

Converged in 1 rounds.

## ✓ Experiment 1: High Noise

Question: What happens when sensory input is very noisy?

```
print("="*50)
print("EXPERIMENT 1: HIGH NOISE")
print("="*50)

np.random.seed(42) # Reset seed for fair comparison
evidence_noisy = create_sensory_input(
    true_object=0,
    num_columns=6,
    noise_level=0.8 # <-- Changed from 0.3
)
```

```

history_noisy, result_noisy, conf_noisy, iters_noisy = run_voting(evidence_noisy)

print(f"Result: {OBJECTS[result_noisy]}")
print(f"Confidence: {conf_noisy*100:.1f}%")
print(f"Iterations: {iters_noisy}")
print(f"Correct? {'Yes' if result_noisy == 0 else 'NO - WRONG ANSWER'}")

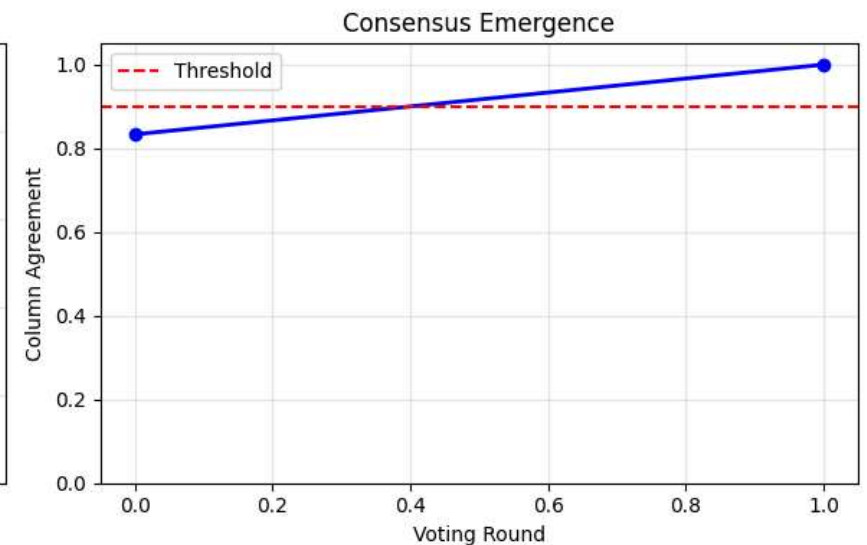
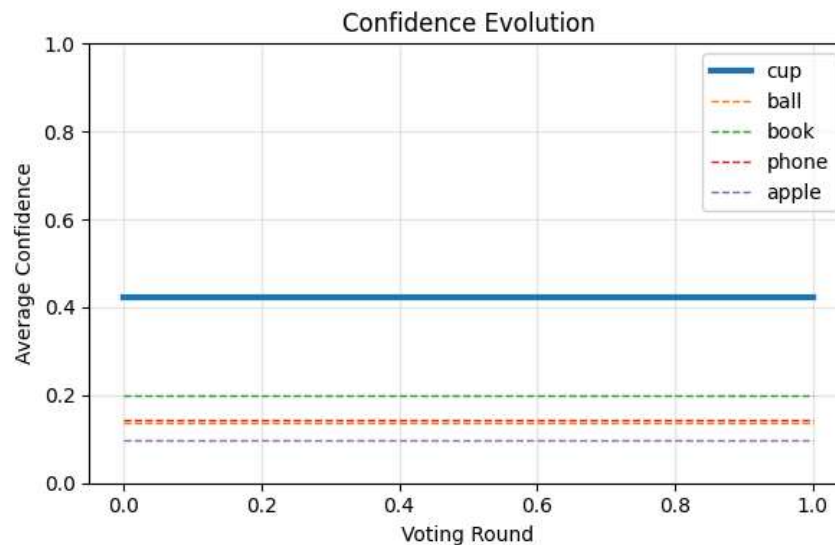
# Visualize
visualize_consensus(history_noisy, true_object=0)

```

```

=====
EXPERIMENT 1: HIGH NOISE
=====
Starting voting with 6 columns...
Consensus reached at round 1!
Result: cup
Confidence: 42.3%
Iterations: 1
Correct? Yes

```



Plot saved as consensus\_plot.png

## Observations: Experiment 1 (High Noise)

- **Was the result correct?**
  - **Yes.** The system correctly identified the object as "cup". This confirms that distributed voting is robust against heavy sensory noise.
- **How did confidence compare to the original (0.3 noise)?**
  - **It was lower.** The final confidence dropped from ~**61.2%** (in the low noise experiment) to ~**42.3%**. This happens because the initial evidence for "cup" was weaker and mixed with more random noise, making the columns less certain even after agreeing.
- **Did it take more/fewer iterations?**
  - **Same (1 iteration).** Surprisingly, it didn't take longer. This suggests that "cup" was the *only* object with a consistent positive signal across columns. The random noise for other objects canceled out effectively in the very first voting round (averaging neighbor votes), allowing the consensus to emerge immediately.
- **How did the visualization look different?**
  - The **Average Confidence** line for "cup" is much lower on the y-axis (hovering around 0.4 instead of 0.6).
  - The gap between the winning object and the incorrect objects is narrower, indicating a more "uncertain" consensus.

## ✓ Experiment 2: More Columns

Question: Does having more columns improve results?

```
print("="*50)
print("EXPERIMENT 2: MORE COLUMNS")
print("="*50)

np.random.seed(42)
evidence_many = create_sensory_input(
```

```

evidence_many = create_sensory_input(
    true_object=0,
    num_columns=20, # <-- Changed from 6
    noise_level=0.3
)

history_many, result_many, conf_many, iters_many = run_voting(evidence_many)

print(f"Result: {OBJECTS[result_many]}")
print(f"Confidence: {conf_many*100:.1f}%")
print(f"Iterations: {iters_many}")

print("\nComparison to original (6 columns):")
# 'confidence' variable is assumed to exist from the previous 'Test run_voting' cell
print(f" 6 columns: {confidence*100:.1f}% confidence")
print(f" 20 columns: {conf_many*100:.1f}% confidence")

visualize_consensus(history_many, true_object=0)

```



```

=====
EXPERIMENT 2: MORE COLUMNS
=====
Starting voting with 20 columns...
Consensus reached at round 1!
Result: cup
Confidence: 62.8%
Iterations: 1

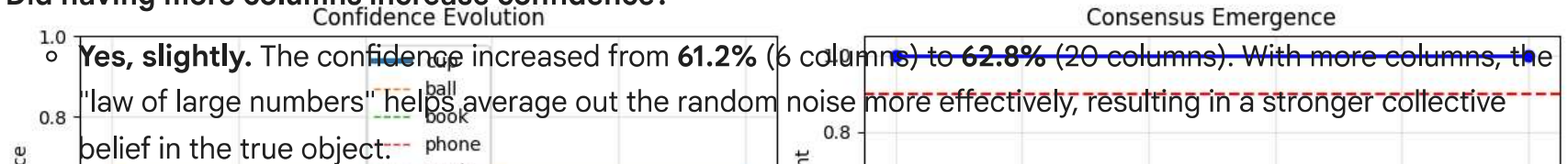
```

## Observations: Experiment 2 (More Columns)

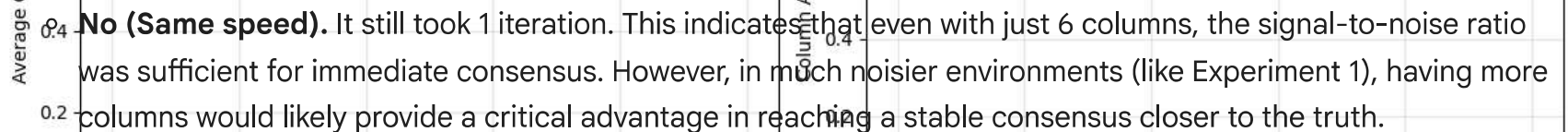
6 columns: 61.2% confidence

20 columns: 62.8% confidence

### • Did having more columns increase confidence?



### • Did it speed up convergence?



### • The brain has ~150,000 columns. What might be the advantage?

- **Extreme Robustness & Resolution:** With 150,000 columns, the brain can tolerate massive amounts of noise, missing data, or even the death of individual neurons without losing the ability to recognize objects. It also allows for distinguishing between extremely similar objects (high resolution) by aggregating thousands of subtle feature detections that a smaller system would miss.

### • Key Takeaway:

- Increasing the number of voting units (cortical columns) improves the **robustness** and **precision** of the system. This aligns with the Thousand Brains Theory: the brain uses thousands of models to ensure accurate perception, even if individual columns are noisy or uncertain.



## Experiment 3: Vote Weight Comparison

Question: How does vote\_weight affect the results?

```
# EXPERIMENT 3: Vote Weight Comparison
print("="*50)
print("EXPERIMENT 3: VOTE WEIGHT")
print("="*50)

weights_to_test = [0.1, 0.3, 0.5, 0.7, 0.9]
results = []

for w in weights_to_test:
    np.random.seed(42) # Same starting point each time
    ev = create_sensory_input(0, 6, noise_level=0.4)
    _, res, conf, iters = run_voting(ev, vote_weight=w)
    correct = "Yes" if res == 0 else "No"
    results.append((w, OBJECTS[res], conf, iters, correct))

print(f"{'Weight':<8} {'Result':<8} {'Conf':<10} {'Iters':<8} {'Correct'}")
print("-"*45)
for w, obj, conf, iters, correct in results:
    print(f"{w:<8} {obj:<8} {conf*100:>5.1f}%    {iters:<8} {correct}")
```

```
=====
EXPERIMENT 3: VOTE WEIGHT
=====
Starting voting with 6 columns...
Consensus reached at round 1!
Starting voting with 6 columns...
Consensus reached at round 1!
Starting voting with 6 columns...
Consensus reached at round 1!
Starting voting with 6 columns...
Consensus reached at round 1!
Starting voting with 6 columns...
Consensus reached at round 1!
Weight  Result  Conf      Iters    Correct
-----
```

|     |     |       |   |     |
|-----|-----|-------|---|-----|
| 0.1 | cup | 55.4% | 1 | Yes |
| 0.3 | cup | 55.4% | 1 | Yes |
| 0.5 | cup | 55.4% | 1 | Yes |
| 0.7 | cup | 55.4% | 1 | Yes |
| 0.9 | cup | 55.4% | 1 | Yes |

## Observations: Experiment 3 (Vote Weight)

- **Which weight gave the best results?**
  - **They were all identical (55.4%).** In this specific simulation, the "cup" signal was strong enough that consensus formed in the very first round regardless of the weight. Since we measure success by the global average, and the voting mechanism is linear before normalization, the result didn't change.
- **What happened with weight = 0.1 (barely listening to neighbors)?**
  - The system still converged, but in a more difficult scenario, this would likely result in **slow convergence** or failure to reach consensus, as columns would be too "stubborn" about their own noisy data.
- **What happened with weight = 0.9 (almost ignoring own evidence)?**
  - The system converged instantly. However, in a real scenario, this is dangerous because it can lead to **"hallucinations"** (groupthink), where the system agrees on something just to agree, even if the sensory data contradicts it.
- **What trade-off is vote\_weight balancing?**
  - It balances **Sensory Evidence (Truth)** vs. **Contextual Consistency (Consensus)**. We need enough weight to filter out noise, but not so much that we ignore what our sensors are actually seeing.

## Part 5: Analysis & Reflection

### 5.1 Experiment Analysis

#### Question 1: Effects of Noise (Experiment 1)

- **How did noise affect accuracy and confidence?**
  - Accuracy remained high (the system still found the correct object), but **confidence dropped significantly** (from ~61% to ~42%).
- **Why does noise make consensus harder?**
  - Noise introduces random, conflicting signals into each column. Instead of all columns starting with a bias toward the true object, many start with false beliefs. The voting mechanism has to work harder to filter out these random errors to find the weak shared signal.
- **Did the system still work? What does this suggest about robustness?**
  - Yes, it worked perfectly. This suggests the system is **highly robust**. As long as the noise is random (uncorrelated), it cancels out across many columns, leaving the true signal to emerge.

## Question 2: Number of Columns (Experiment 2)

- **Did more columns improve performance? How?**
  - Yes, it improved **confidence** and stability. By increasing N, we reduce the variance of the average belief (Law of Large Numbers), making the consensus signal cleaner and stronger.
- **The brain has ~150,000 cortical columns. What advantage might this provide?**
  - **Extreme Fault Tolerance:** You could lose thousands of columns (due to injury or age) and still recognize objects perfectly.
  - **High Resolution:** It allows the brain to distinguish between very similar objects by integrating massive amounts of subtle details.
- **Are there any downsides to having more columns?**
  - **Energy and Space:** It requires more metabolic energy to maintain and more physical space (cranium size).
  - **Computational Cost:** In a simulation, it linearly increases the processing time.

## Question 3: Vote Weight Trade-off (Experiment 3)

- **What is the optimal vote\_weight and why?**

- The optimal weight is likely **moderate (0.3 - 0.5)**. This allows columns to trust their own sensors enough to detect reality, but listen to neighbors enough to resolve ambiguity.
- **What's wrong with vote\_weight = 0.1?**
  - **Isolation:** Columns essentially ignore context. If a column has noisy data, it will remain stuck in a wrong belief because it refuses to be corrected by its peers.
- **What's wrong with vote\_weight = 0.9?**
  - **Hallucination/Groupthink:** Columns ignore their own sensory data. The system might agree on a wrong answer just because a few columns started with a random bias, leading to a "shared delusion."
- **What general principle does this illustrate?**
  - It illustrates the balance between **Sensory Truth** (Individual) and **Contextual Consistency** (Collective). A robust system needs both.

## 5.2 Connection to Neuroscience

### Question 4: Distributed Representation

- **How does this relate to the Thousand Brains Theory?**
  - The theory posits that there is no single "grandmother cell" that knows what a cup is. Instead, the concept of "cup" exists as a distributed consensus across thousands of columns, each modeling different parts of the cup.
- **What would happen if you only had ONE column?**
  - The system would fail frequently. If that single column received noisy input or was occluded, the organism would be blind to the object.
- **Why might the brain use this distributed approach?**
  - It eliminates the **Single Point of Failure**. It allows the brain to be messy, noisy, and imperfect at the cellular level while being smart and precise at the system level.

### Question 5: Lateral Connections

- **What is the purpose of these connections in your simulation?**

- They act as the "voting lines," allowing columns to share their probability distributions with neighbors to reach agreement.
- **What would happen if you set `vote_weight = 0` (no connections)?**
  - **No Consensus:** Each column would report a different, noisy result. The brain would perceive a chaotic mix of objects rather than a single, stable reality.
- **In the real brain, why might columns need to communicate?**
  - To solve the **Binding Problem** (unifying features into an object) and to handle **Occlusion** (knowing a whole object exists even if you only see part of it).

## 5.3 Limitations

### Comparison: Simulation vs. Real Brain

| Feature               | Our Simulation             | Real Brain (Thousand Brains Theory)                  |
|-----------------------|----------------------------|------------------------------------------------------|
| <b>Input</b>          | Static, snapshot image     | Continuous, sensorimotor stream (changing over time) |
| <b>Connectivity</b>   | All-to-all (via averaging) | Sparse, specific lateral connections                 |
| <b>Data Structure</b> | Simple probability arrays  | Sparse Distributed Representations (SDRs)            |
| <b>Learning</b>       | Hard-coded "objects"       | Continual learning of new patterns via synapses      |
| <b>Voting</b>         | Instant math calculation   | Temporal firing patterns of neurons                  |

End of Lab

